



به نام خدا

دانشگاه تهران



دانشکده مهندسی برق و کامپیوتر

درس شبکه‌های عصبی و یادگیری عمیق تمرین اول

نام و نام خانوادگی	سید مانی میرشعبانی	پرسش ۱
شماره دانشجویی	810801080	
نام و نام خانوادگی	سید مانی میرشعبانی	پرسش ۲
شماره دانشجویی	810801080	

فهرست

قوانین.....	1
پرسش 1. طبقه بندی تصاویر با ViT.....	3
۱-۱. مقدمه.....	3
۱-۱-۱. مزیت استفاده از مدل های ویژن ترنسفورمر در مقایسه با مدل های سنتی چیست؟.....	3
۱-۱-۲. کدامیک در شرایط داده محدود بهتر عمل می کند؟.....	3
۱-۲. آماده سازی داده ها.....	4
۱-۲-۱. دانلود و نمایش داده ها.....	4
۱-۲-۲. بررسی عدم تعادل کلاس ها، افزایش داده (Augmentaion)، تغییر اندازه تصاویر و تقسیم دیتاست.....	5
۱-۳. آموزش مدل CNN (InceptionV3).....	7
۱-۳-۱. نحوه عملکرد مدل Inception V3.....	8
۱-۳-۲. تابع هزینه.....	9
۱-۴. آموزش مدل ViT.....	12
۱-۴-۱. پیاده سازی مدل مقاله.....	12
۱-۴-۲. Patch Embedding.....	17
۱-۴-۳. آموزش مدل ViT.....	18
۱-۵. تحلیل و نتیجه گیری.....	20
پرسش ۲ - Robust Zero-Shot Classification.....	23
۲-۰. مقدمه:.....	23
۲-۱. آشنایی با مدل CLIP، طبقه بندی تک ضرب و حملات خصمانه.....	23
۲-۱-۲. FGSM و PGD.....	23
۲-۱-۲. CLIP.....	25
۲-۱-۳. طبقه بندی تک ضرب در مقابل طبقه بندی معمولی.....	28
۲-۱-۴. انواع حملات خصمانه.....	30
۲-۱-۵. حملات انتقالی.....	31
۲-۱-۶. LoRA.....	31
۲-۱-۷. دو مقاله که CLIP را گسترش می دهند.....	33

۲-۲. پیاده سازی و مقایسه روش های آموزش خصمانه.....	34
۲-۲-۱. آماده سازی داده ها.....	34
۲-۲-۲. آماده سازی مدل.....	35
۲-۲-۳. آزمایش CLIP، حمله PGD و بصری سازی.....	35
۲-۲-۴. LoRA.....	36
۲-۲-۵. TeCoA.....	37
۲-۲-۶. مقایسه و ارزیابی.....	38

شکل‌ها

- شکل 1: تصویر هر یک از ۱۰ کلاس دیتاست.....5
- شکل 2: توضیح کلاس‌ها.....6
- شکل 3: تصاویر Augment شده.....7
- شکل 4: معماری InceptionV3.....9
- شکل 5: رابطه Focal Loss.....9
- شکل 6: رابطه KL Divergence.....10
- شکل 7: خلاصه مدل.....10
- شکل 8: نمودار دقت و تابع هزینه InceptionV3.....11
- شکل 9: ماتریس آشفتگی InceptionV3.....12
- شکل 10: تعداد پارامترهای مدل ViT.....17
- شکل 11: نمودار دقت و تابع هزینه مدل ViT.....18
- شکل 12: ماتریس آشفتگی ViT.....19
- شکل 13: مقایسه عملکرد کلی مدل‌ها.....20
- شکل 14: جزئیات عملکرد مدل CNN از پیش آموزش دیده.....20
- شکل 15: جزئیات مدل CNN بدون آموزش قبلی.....21
- شکل 16: جزئیات عملکرد مدل ViT.....21
- شکل 17: فرمول محاسبه FGSM.....24
- شکل 18: خلاصه معماری CLIP.....26
- شکل 19: شبه کد پیاده سازی loss برای CLIP.....27
- شکل 20: طبقه بندی تک ضرب در CLIP.....29
- شکل 21: خلاصه عملکرد LoRA.....32
- شکل 22: نمونه تصاویر CIFAR-10.....35
- شکل 23: دقت CLIP روی داده های تمیز ۸۷.۸۳٪.....36
- شکل 24: دقت CLIP روی داده های خصمانه ۸۴.۵۶٪.....36
- شکل 25: نمونه داده اصلی، داده خصمانه و نویز اضافه شده (برای نمایش بهتر نویز شدت آن ۱۰ برابر شده است).....36
- شکل 26: دقت بعد از Fine-tuning با LoRA.....37
- شکل 27: دقت بعد از Fine-tuning با TeCoA.....38

شکل 28: مقایسه عملکرد هر دو روش.....38

جدول‌ها

جدول 1: خلاصه مدل ViT 13

جدول 2: مقایسه روشهای Black-Box و White-Box 30

قبل از پاسخ دادن به پرسش‌ها، موارد زیر را با دقت مطالعه نمایید:

- از پاسخ‌های خود یک گزارش در قالبی که در صفحه‌ی درس در سامانه‌ی Elearn با نام [REPORTS_TEMPLATE.docx](#) قرار داده شده تهیه نمایید.
- پیشنهاد می‌شود تمرین‌ها را در قالب گروه‌های دو نفره انجام دهید. (بیش از دو نفر مجاز نیست و تحویل تک نفره نیز نمره‌ی اضافی ندارد) توجه نمایید الزامی در یکسان ماندن اعضای گروه تا انتهای ترم وجود ندارد. (یعنی، می‌توانید تمرین اول را با شخص A و تمرین دوم را با شخص B و ... انجام دهید)
- **کیفیت گزارش شما در فرآیند تصحیح از اهمیت ویژه‌ای برخوردار است؛** بنابراین، لطفاً تمامی نکات و فرضیه‌هایی را که در پیاده‌سازیها و محاسبات خود در نظر می‌گیرید در گزارش ذکر کنید.
- در گزارش خود مطابق با آنچه در قالب نمونه قرار داده شده، برای شکل‌ها زیرنویس و برای جدول‌ها بالانویس در نظر بگیرید.
- الزامی به ارائه توضیح جزئیات کد در گزارش نیست، اما باید نتایج بدست آمده از آن را گزارش و تحلیل کنید.
- **تحلیل نتایج الزامی می‌باشد، حتی اگر در صورت پرسش اشاره‌ای به آن نشده باشد.**
- **دستیاران آموزشی ملزم به اجرا کردن کدهای شما نیستند؛** بنابراین، هرگونه نتیجه و یا تحلیلی که در صورت پرسش از شما خواسته شده را به طور واضح و کامل در گزارش بیاورید. در صورت عدم رعایت این مورد، بدیهی است که از نمره تمرین کسر میشود.
- **کدها حتماً باید در قالب نوت‌بوک با پسوند .ipynb تهیه شوند، در پایان کار، تمامی کد اجرا شود و خروجی هر سلول حتماً در این فایل ارسالی شما ذخیره شده باشد.** بنابراین برای مثال اگر خروجی سلولی یک نمودار است که در گزارش آورده‌اید، این نمودار باید هم در گزارش هم در نوت‌بوک کدها وجود داشته باشد.
- **در صورت مشاهده‌ی تقلب امتیاز تمامی افراد شرکت‌کننده در آن، 100- لحاظ میشود.**
- تنها زبان برنامه نویسی مجاز Python است.
- **استفاده از کدهای آماده برای تمرینها به هیچ وجه مجاز نیست.**
- **در صورتی که دو گروه از یک منبع مشترک استفاده کنند و کدهای مشابه تحویل دهند، تقلب محسوب می‌شود.**

- نحوه محاسبه تاخیر به این شکل است: پس از پایان رسیدن مهلت ارسال گزارش، حداکثر تا یک هفته امکان ارسال با تاخیر وجود دارد، پس از این یک هفته نمره آن تکلیف برای شما صفر خواهد شد.

0 سه روز اول: بدون جریمه

0 روز چهارم: ۵ درصد

0 روز پنجم: ۱۰ درصد

0 روز ششم: ۱۵ درصد

0 روز هفتم: ۲۰ درصد

- حداکثر نمره‌ای که برای هر سوال می‌توان اخذ کرد ۱۰۰ بوده و اگر مجموع بارم یک **سوال** بیشتر از ۱۰۰ باشد، در صورت اخذ نمره بیشتر از ۱۰۰، اعمال نخواهد شد.

0 برای مثال: اگر نمره اخذ شده از سوال ۱ برابر ۱۰۵ و نمره سوال

۲ برابر ۹۵ باشد، نمره نهایی تمرین ۹۷.۵ خواهد بود و نه ۱۰۰.

- لطفا گزارش، کدها و سایر ضمایم را به در یک پوشه با نام زیر قرار داده و آن را فشرده سازید، سپس در سامانه‌ی Elearn بارگذاری نمایید:

HW[Number]_[Lastname]_[StudentNumber]_[Lastname]_[StudentNumber].zip

(مثال: HW1_Ahmadi_810199101_Bagheri_810199102.zip)

- برای گروه‌های دو نفره، بارگذاری تمرین از جانب یکی از اعضا کافی است ولی پیشنهاد می‌شود هر دو نفر بارگذاری نمایند.

پرسش 1. طبقه بندی تصاویر با ViT

۱-۱. مقدمه

در این تمرین، یک ویژن ترانسفورمر پیاده سازی خواهیم کرد و آن را با یک شبکه عصبی CNN روی مجموعه داده ای از برگ های گوجه فرنگی با بیماری های مختلف مقایسه می کنیم.

ابتدا به چند سؤال زیر پاسخ دهیم:

۱-۱-۱. مزیت استفاده از مدل های ویژن ترانسفورمر در

مقایسه با مدل های سنتی چیست؟

بر اساس مقاله، تفاوت اصلی مدل های ویژن ترانسفورمر (ViT) در مقایسه با مدل هایی مانند CNN استفاده از مکانیزم توجه (Attention) به جای عملیات کانولوشن است. در حالی که شبکه های عصبی CNN عمدتاً بر عملیات کانولوشن متکی هستند، مدل های ViT تنها بر مکانیزم های توجه (Self-Attention) استوارند. این مدل ها تصویر را به توالی ای از تکه ها (patches) تقسیم می کنند و روابط بین آن ها را تحلیل می کنند، فرآیندی که به کانولوشن نیاز ندارد.

مزیت کلیدی ViT در توانایی پردازش مؤثرتر ویژگی هاست. بر خلاف مدل های سنتی که ممکن است اهمیت ویژگی های مختلف را یکسان فرض کنند، ViT با مکانیزم توجه اش به صورت دینامیک وزن های متفاوتی به بخش های مختلف تصویر ورودی اختصاص می دهد و این امکان را می دهد که مدل به طور خودکار روی ویژگی های مرتبط تر متمرکز شود و از نادیده گرفتن جزئیات مهم هنگام طبقه بندی جلوگیری کند. مقاله نشان می دهد که مدل ViT در دقت بالاتری نسبت به مدل Inception V3 سنتی به دست آورده.

۱-۱-۲. کدامیک در شرایط داده محدود بهتر عمل می کند؟

بر اساس مقاله و با توجه به ساختار مدل ها، مدل سنتی مانند CNN (Inception V3) در شرایطی که مجموعه داده محدود است، به دلایل زیر عملکرد بهتری دارد:

- CNN ها و Inductive Bias: شبکه های کانولوشنی دارای سوگیری های مهمی در ساختار خود هستند، به عنوان مثال Translation Equivariance که از عملیات کانولوشن ناشی می شود. این یعنی یک CNN می تواند یک شی را صرف نظر از مکانش در تصویر تشخیص دهد. این باعث می شود مدل به داده کمتری نیاز داشته باشد، چون نیازی نیست این مفهوم را از ابتدا با تعداد زیادی مثال یاد بگیرد.

- وابستگی ViT به داده های بزرگ: مقاله بیان می کند که معماری ویژن ترنسفورمر «در وظایف طبقه بندی تصویر در مقیاس وسیع عملکرد فوق العاده ای دارد» که نشان می دهد قدرت آن زمانی آشکار می شود که بتواند از مقدار زیادی داده بیاموزد، که با تعداد زیاد پارامتر در مدل ترنسفورمر همخوانی دارد.

- مزیت یادگیری انتقالی برای CNN ها: مقاله همچنین اشاره می کند که Inception V3 بر روی مجموعه داده های بزرگی مانند ImageNet از پیش آموزش دیده است. این امر به استخراج ویژگی های معنادار کمک می کند و با استفاده از تکنیک یادگیری انتقالی، مدل می تواند آن دانش قبلی خود را به مجموعه داده کوچکتر و جدید با کارایی بالا انتقال دهد.

۱-۲. آماده سازی داده ها

۱-۲-۱. دانلود و نمایش داده ها

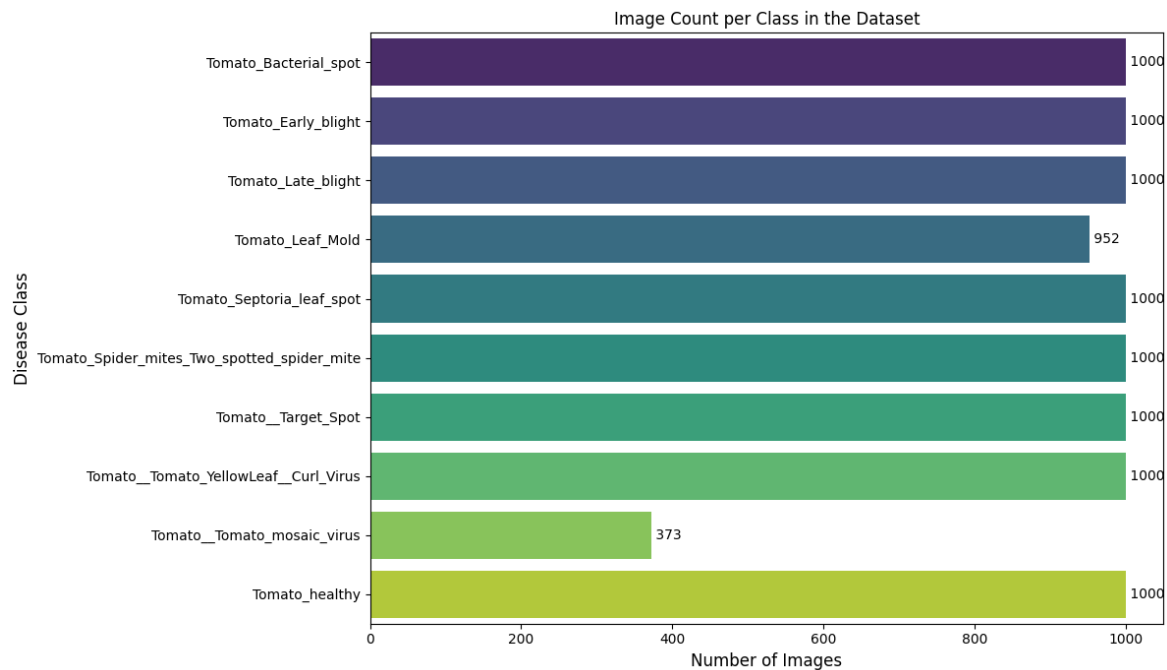
در گام نخست داده ها را از Hugging Face دانلود کرده و یک تصویر از هر یک از ده کلاس نمایش می دهیم.



شکل 1: تصویر هر یک از ۱۰ کلاس دیتاست

۱-۲-۲. بررسی عدم تعادل کلاس‌ها، افزایش داده (Augmentation)، تغییر اندازه تصاویر و تقسیم دیتاست

در این بخش ابتدا عدم تعادل کلاس‌ها در داده‌ها را بررسی می‌کنیم، سپس یک راه حل برای رفع آن پیشنهاد داده و آن را پیاده‌سازی می‌کنیم و داده‌ها را به مجموعه آموزش و مجموعه اعتبارسنجی تقسیم می‌کنیم. برای بررسی عدم تعادل کلاس‌ها، ابتدا مجموعه داده را به یک DataFrame از pandas تبدیل می‌کنیم و سپس یک نمودار میله‌ای برای نمایش توزیع کلاس‌ها رسم می‌کنیم.



شکل 2: توزیع کلاس ها

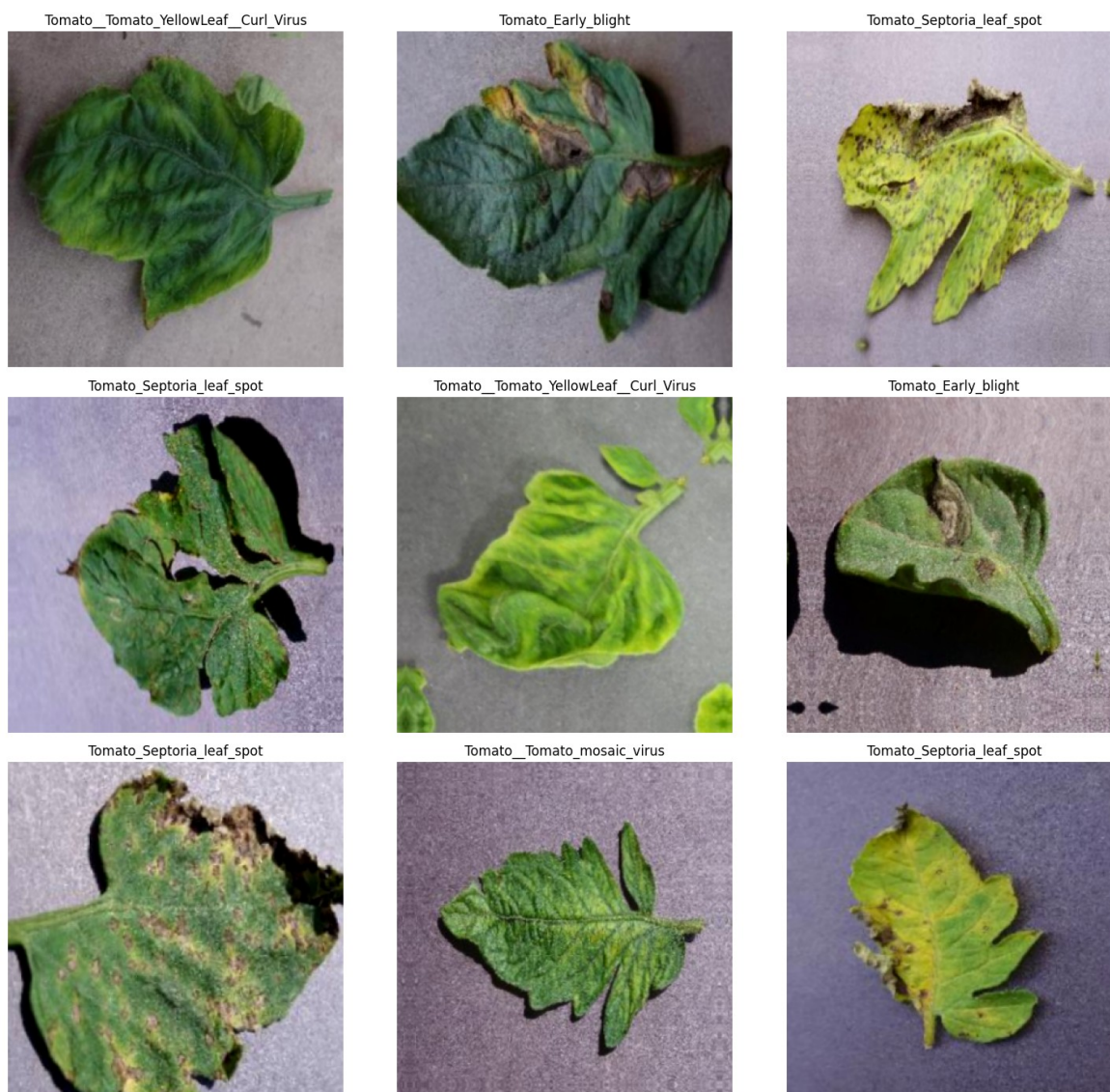
از نمودار مشاهده می‌شود که تنها دو کلاس کمتر از ۱۰۰۰ نمونه دارند: Tomato_Leaf_Mold با ۹۵۲ نمونه و Tomato__Tomato_mosaic_virus با تنها ۳۷۳ نمونه.

برای حل این مشکل سه گزینه داریم: نمونه‌برداری کمتر از کلاس‌های دیگر، نمونه‌برداری مضاعف (Oversampling) از کلاس‌های با نمونه کم یا افزایش داده (augmentation). از آنجایی که دو روش اول ممکن است عمل‌کرد مدل را کاهش دهند، از روش افزایش داده استفاده می‌کنیم همچنین برای کارایی بهتر، یک Augmenter تعریف می‌کنیم (و داده‌های Augment شده را ذخیره نمی‌کنیم!) و نمونه‌های این دو کلاس را بیشتر از کلاس‌های دیگر افزایش می‌دهیم تا به مجموعه‌ای متوازن‌تر برسیم. (در اصل از هر ۱۰ کلاس به اندازه برابر سмпل می‌کنیم که این کار باعث می‌شود هنگام آموزش داده‌های بیشتری برای این دو کلاس تولید شوند)

برای افزایش داده، تبدیل‌های Flip افقی و عمودی، چرخش، زوم و تنظیم کنتراست تعریف کردیم چرا که در واقعیت همه این‌ها ممکن هستند. توجه داشته باشید که به دلیل اندازه‌های ورودی متفاوت مدل‌های ViT و Inception V ۳، دو مجموعه داده مجزا با ابعاد تصویر ۶۴×۶۴ و ۲۹۹×۲۹۹ ایجاد کرده‌ایم و برای هر مدل دو دسته داریم: یک مجموعه برای آموزش و یک مجموعه برای اعتبارسنجی (بدون افزایش داده روی مجموعه اعتبارسنجی). مشابه مقاله، ۹۰٪ داده‌ها برای آموزش و ۱۰٪ برای اعتبارسنجی استفاده می‌شود.

حال چند تصویر را نمایش می‌دهیم تا اطمینان حاصل کنیم همه مراحل به درستی انجام شده است.

Sample Augmented Images from a Training Batch



شکل 3: تصاویر Augment شده

۱-۳. آموزش مدل CNN (InceptionV3)
در این بخش مدل InceptionV3 را آموزش می‌دهیم، اما پیش از شروع توضیحی کوتاه درباره معماری این مدل ارائه می‌کنیم.

۱-۳-۱. نحوه عملکرد مدل Inception V3

Inception V3 یک معماری یادگیری عمیق و نوعی شبکه CNN است. که از لایه‌های کانولوشن، پولینگ و fully connected تشکیل شده و هدف اصلی آن طبقه‌بندی تصویر با کارایی بالا و محاسبات بهینه است.

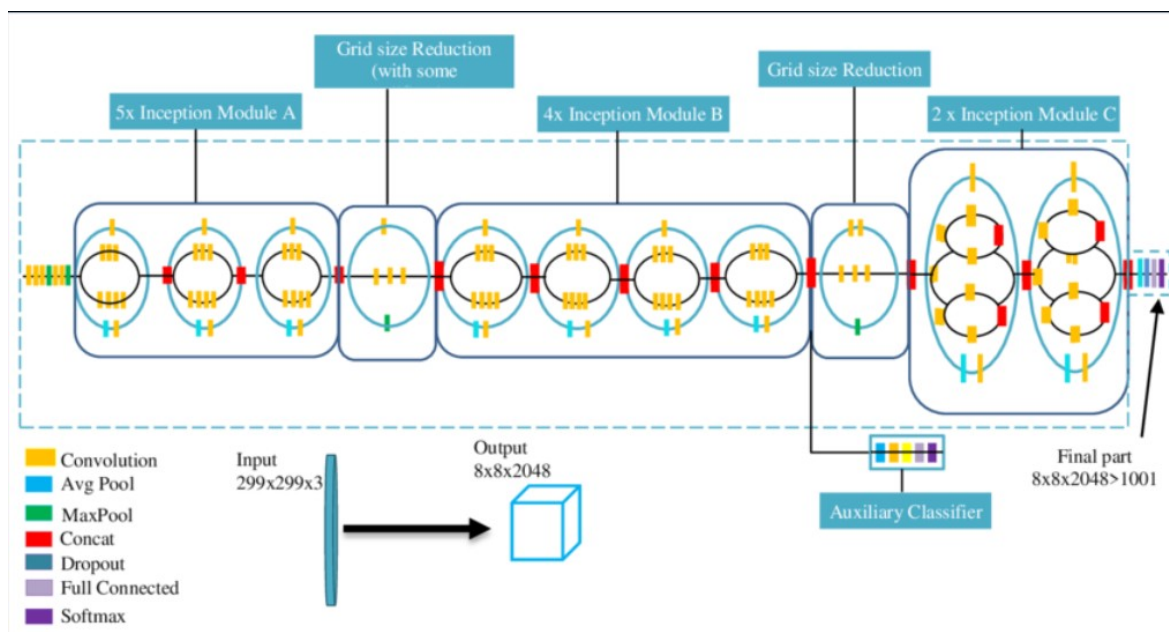
نوآوری کلیدی Inception در بلوک ساختار اصلی آن، یعنی ماژول Inception است که از بخش‌های زیر تشکلی شده است:

۱. استخراج ویژگی‌ها در مقیاس‌های مختلف: ماژول Inception با اجرای کانولوشن‌هایی با اندازه فیلتر متفاوت به‌طور موازی روی ورودی، همزمان ویژگی‌های کوچک و بزرگ تصویر را شناسایی می‌کند.

۲. پردازش موازی: ورودی ماژول از چند شاخه موازی عبور می‌کند و خروجی تمام شاخه‌ها پس از پردازش با هم ادغام (concatenate) می‌شود تا یک ویژگی غنی تولید شود.

۳. طبقه‌بندی نهایی: پس از عبور تصویر از تمامی ماژول‌های Inception، ویژگی‌های استخراج‌شده توسط یک classifier نهایی پردازش و نتیجه طبقه‌بندی ارائه می‌شود.

این بخش‌ها در شکل زیر قابل مشاهده هستند:



شکل 4: معماری InceptionV3

۱-۳-۲. تابع هزینه

در مقاله استفاده از categorical cross-entropy به عنوان تابع هزینه برای آموزش مدل‌ها پیشنهاد شده است که یک تابع استاندارد در مسائل طبقه‌بندی چندکلاسه است.

علاوه بر کراس‌انتروپی، توابع دیگری نیز قابل استفاده هستند: ۱. Focal Loss: نسخه‌ای بهبودیافته از کراس‌انتروپی است که با کاهش وزن خطاهای ساده و افزایش توجه به نمونه‌های دشوارتر، عملکرد بهتری در شرایط عدم تعادل کلاس‌ها ارائه می‌دهد.

$$Focal Loss = - \sum_{i=1}^{i=n} (i - p_i)^{\gamma} \log_b(p_i)$$

شکل 5: رابطه Focal Loss

۲. Kullback–Leibler (KL Divergence): معیاری برای سنجش تفاوت دو توزیع احتمال است. برای برچسب‌های one-hot در مسائل چندکلاسه میتوان نشان داد که معادل کراس‌انتروپی می‌باشد و نتایج مشابهی تولید می‌کند.

$$KL(P||Q) = \frac{\sum_x P(x) \log \frac{P(x)}{Q(x)}}{n}$$

شکل 6: رابطه KL Divergence

۳-۱-۳. آموزش مدل

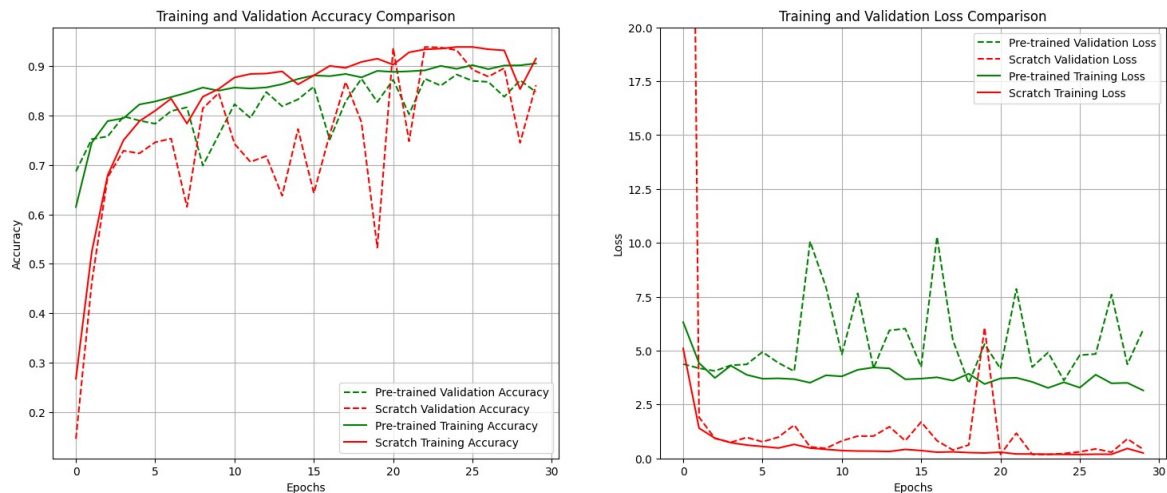
مقاله بیان کرده است که مدل InceptionV3 با وزن‌های از پیش‌آموزش‌دیده روی ImageNet استفاده شده است، اما ما هم نسخه بدون پیش‌آموزش و هم نسخه پیش‌آموزش‌دیده را برای ۳۰ اپیاک آموزش می‌دهیم و نتایج آن‌ها را با هم مقایسه خواهیم کرد.

Layer (type)	Output Shape	Param #
inception_v3 (Functional)	(None, 8, 8, 2048)	21,802,784
flatten_1 (Flatten)	(None, 131072)	0
dense_1 (Dense)	(None, 10)	1,310,730

شکل 7: خلاصه مدل

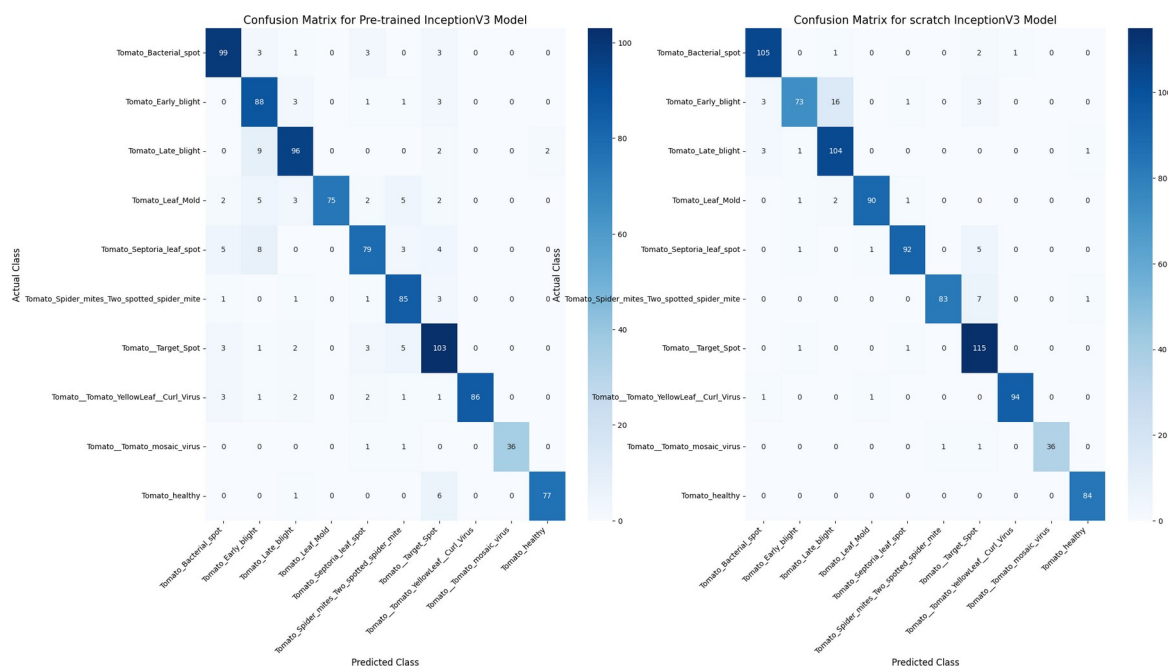
مدل از پیش‌آموزش‌دیده موفق شد به دقت ۰٫۸۸۳۱۷ روی مجموعه اعتبارسنجی دست یابد و مدل آموزش‌داده‌شده از ابتدا (scratch) توانست به دقت ۰٫۹۳۸۹۱ برسد.

در ادامه نمودارهای دقت و خطای آموزش و اعتبارسنجی برای هر دو مدل را مشاهده می‌کنیم.



شکل 8: نمودار دقت و تابع هزینه InceptionV3

از نمودارها مشخص است که مدل از پیش‌آموزش‌دیده دقت کمی پایین‌تر (و خطایی بیشتر) داشته و مدلی که از ابتدا آموزش دید، با وجود شروع ضعیف‌تر، در نهایت توانست به دقتی کمی بالاتر دست پیدا کند، اما عملکرد کلی آن روی داده اعتبارسنجی نوسان زیادی داشت و بالا و پایین‌های زیادی را تجربه کرد.



شکل 9: ماتریس آشفتگی InceptionV3

از ماتریس‌های آشفتگی (confusion matrix) نیز مشاهده می‌شود که هر دو مدل به طور کلی عملکرد خوبی داشته‌اند. مدل از پیش‌آموزش‌دیده خطاهای پراکنده‌تری داشته، در حالی که اکثر خطاهای مدل scratch مربوط به کلاس "Tomato_Early_blight" بودند.

۱-۴. آموزش مدل ViT

۱-۴-۱. پیاده‌سازی مدل مقاله

مدل ViT مقاله با تغییرات زیر پیاده‌سازی شده است:

- ایجاد Patch ها قبل از MLP: در متن مقاله آمده است که «A network that can change the images into patches was introduced after the MLP layer» که نادرست است. ایجاد patch ها اولین گام در ViT است و MLP در بلاک‌های ترنسفورمر قرار دارد.

- ابعاد ترنسفورمر متفاوت: در جدول ۳ «Transformer Dim» برابر با ۲ ذکر شده است. ابعاد پروژکشن ۲ باعث می‌شود تمام اطلاعات یک patch به تنها دو عدد خلاصه شوند که یادگیری را غیرممکن می‌کند. به احتمال زیاد این قسمت مقاله اشتباه است بنابراین ما از عدد ۶۴ استفاده کردیم که با ابعاد خروجی لایه‌ها در جدول ۴ مقاله همخوانی دارد.

• خلاصه مدل ناقص: جدول ۴ مقاله قرار بوده خلاصه مدل باشد اما لایه‌های Multi-Head Attention را حذف کرده است. این لایه‌ها هسته معماری ترنسفورمر هستند. پیاده‌سازی ما آن‌ها را شامل می‌شود.

• پارامتر لایه‌های پروجکشن متفاوت: در جدول ۳ «Projection Layers» برابر با ۸ ذکر شده است، Projection Layers اصطلاح استاندارد ViT نیست. بعد از مقداری جست‌وجو این عدد را به تعداد بلاک‌های Encoder تفسیر کردیم و در پیاده‌سازی از ۸ بلاک ترنسفورمر استفاده کردیم. سایر پارامترها مطابق مقاله هستند.

جدول 1: خلاصه مدل ViT

# Param	Output Shape	Layer (type)
0	(None, 64, 64, 3)	input_layer_8 (InputLayer)
0	(None, None, 192)	patches_2 (Patches)
16448	(None, 64, 64)	patch_encoder_2 (PatchEncoder)
128	(None, 64, 64)	layer_normalization_32 (LayerNormalization)
66368	(None, 64, 64)	multi_head_attention_24 (MultiHeadAttention)
0	(None, 64, 64)	add_32 (Add)
128	(None, 64, 64)	layer_normalization_33 (LayerNormalization)
8320	(None, 64, 128)	dense_43 (Dense)
0	(None, 64, 128)	dropout_55 (Dropout)
8256	(None, 64, 64)	dense_44 (Dense)
0	(None, 64, 64)	dropout_56 (Dropout)
0	(None, 64, 64)	add_33 (Add)
128	(None, 64, 64)	layer_normalization_34 (LayerNormalization)
66368	(None, 64, 64)	multi_head_attention_25 (MultiHeadAttention)

0	(None, 64, 64)	add_34 (Add)
128	(None, 64, 64)	layer_normalization_35 (LayerNormalization)
8320	(None, 64, 128)	dense_45 (Dense)
0	(None, 64, 128)	dropout_58 (Dropout)
8256	(None, 64, 64)	dense_46 (Dense)
0	(None, 64, 64)	dropout_59 (Dropout)
0	(None, 64, 64)	add_35 (Add)
128	(None, 64, 64)	layer_normalization_36 (LayerNormalization)
66368	(None, 64, 64)	multi_head_attention_26 (MultiHeadAttention)
0	(None, 64, 64)	add_36 (Add)
128	(None, 64, 64)	layer_normalization_37 (LayerNormalization)
8320	(None, 64, 128)	dense_47 (Dense)
0	(None, 64, 128)	dropout_61 (Dropout)
8256	(None, 64, 64)	dense_48 (Dense)
0	(None, 64, 64)	dropout_62 (Dropout)
0	(None, 64, 64)	add_37 (Add)
128	(None, 64, 64)	layer_normalization_38 (LayerNormalization)
66368	(None, 64, 64)	multi_head_attention_27 (MultiHeadAttention)
0	(None, 64, 64)	add_38 (Add)
128	(None, 64, 64)	layer_normalization_39 (LayerNormalization)
8320	(None, 64, 128)	dense_49 (Dense)
0	(None, 64, 128)	dropout_64 (Dropout)
8256	(None, 64, 64)	dense_50 (Dense)
0	(None, 64, 64)	dropout_65 (Dropout)

0	(None, 64, 64)	add_39 (Add)
128	(None, 64, 64)	layer_normalization_40 (LayerNormalization)
66368	(None, 64, 64)	multi_head_attention_28 (MultiHeadAttention)
0	(None, 64, 64)	add_40 (Add)
128	(None, 64, 64)	layer_normalization_41 (LayerNormalization)
8320	(None, 64, 128)	dense_51 (Dense)
0	(None, 64, 128)	dropout_67 (Dropout)
8256	(None, 64, 64)	dense_52 (Dense)
0	(None, 64, 64)	dropout_68 (Dropout)
0	(None, 64, 64)	add_41 (Add)
128	(None, 64, 64)	layer_normalization_42 (LayerNormalization)
66368	(None, 64, 64)	multi_head_attention_29 (MultiHeadAttention)
0	(None, 64, 64)	add_42 (Add)
128	(None, 64, 64)	layer_normalization_43 (LayerNormalization)
8320	(None, 64, 128)	dense_53 (Dense)
0	(None, 64, 128)	dropout_70 (Dropout)
8256	(None, 64, 64)	dense_54 (Dense)
0	(None, 64, 64)	dropout_71 (Dropout)
0	(None, 64, 64)	add_43 (Add)
128	(None, 64, 64)	layer_normalization_44 (LayerNormalization)
66368	(None, 64, 64)	multi_head_attention_30 (MultiHeadAttention)
0	(None, 64, 64)	add_44 (Add)
128	(None, 64, 64)	layer_normalization_45 (LayerNormalization)

8320	(None, 64, 128)	dense_55 (Dense)
0	(None, 64, 128)	dropout_73 (Dropout)
8256	(None, 64, 64)	dense_56 (Dense)
0	(None, 64, 64)	dropout_74 (Dropout)
0	(None, 64, 64)	add_45 (Add)
128	(None, 64, 64)	layer_normalization_46 (LayerNormalization)
66368	(None, 64, 64)	multi_head_attention_31 (MultiHeadAttention)
0	(None, 64, 64)	add_46 (Add)
128	(None, 64, 64)	layer_normalization_47 (LayerNormalization)
8320	(None, 64, 128)	dense_57 (Dense)
0	(None, 64, 128)	dropout_76 (Dropout)
8256	(None, 64, 64)	dense_58 (Dense)
0	(None, 64, 64)	dropout_77 (Dropout)
0	(None, 64, 64)	add_47 (Add)
128	(None, 64, 64)	layer_normalization_48 (LayerNormalization)
0	(None, 4096)	flatten_4 (Flatten)
0	(None, 4096)	dropout_78 (Dropout)
8390656	(None, 2048)	dense_59 (Dense)
0	(None, 2048)	dropout_79 (Dropout)
2098176	(None, 1024)	dense_60 (Dense)
0	(None, 1024)	dropout_80 (Dropout)
10250	(None, 10)	dense_61 (Dense)

Total params: 11,181,258 (42.65 MB)

Trainable params: 11,181,258 (42.65 MB)

Non-trainable params: 0 (0.00 B)

شکل 10: تعداد پارامترهای مدل ViT

۲-۴-۱. Patch Embedding

لایه Patch Embedding گام اول در ViT است که یک ترنسفورمر استاندارد را برای کار با داده‌های تصویری دو بعدی آماده میکند. این لایه سه هدف اصلی دارد:

- ۱. تبدیل تصویر به دنباله: این لایه ابتدا تصویر ورودی را به شبکه‌ای از تکه‌های کوچک تقسیم می‌کند و سپس هر تکه را به یک بردار تبدیل (Flatten) می‌کند. با تخت‌سازی تکه‌ها به بردارهایی با طول یکسان، این فرایند عملاً تصویر دوبعدی را به یک دنباله یک‌بعدی از توکن‌ها تبدیل می‌کند که قالب ورودی ترنسفورمر است.

- ۲. ایجاد embedding های قابل یادگیری: بردارهای تخت‌شده ممکن است ابعاد بسیار بالایی داشته باشند. برای مدیریت بهتر، هر تکه از یک لایه Dense عبور داده می‌شود تا با یک نگاشت خطی ابعاد بالای آن به فضای برداری با ابعاد ثابت و کوچکتر پروجکت شود.

- ۳. افزودن اطلاعات مکانی: معماری استاندارد ترنسفورمر فاقد درک ترتیب توکن‌ها در دنباله ورودی است. برای حل این مسئله، لایه Patch Embedding کدگذاری‌های مکانی قابل یادگیری را به embedding ها اضافه می‌کند. این مرحله به مدل اطلاع می‌دهد که هر تکه در کجای تصویر قرار داشته است و به درک روابط فضایی بین آن‌ها کمک می‌کند.

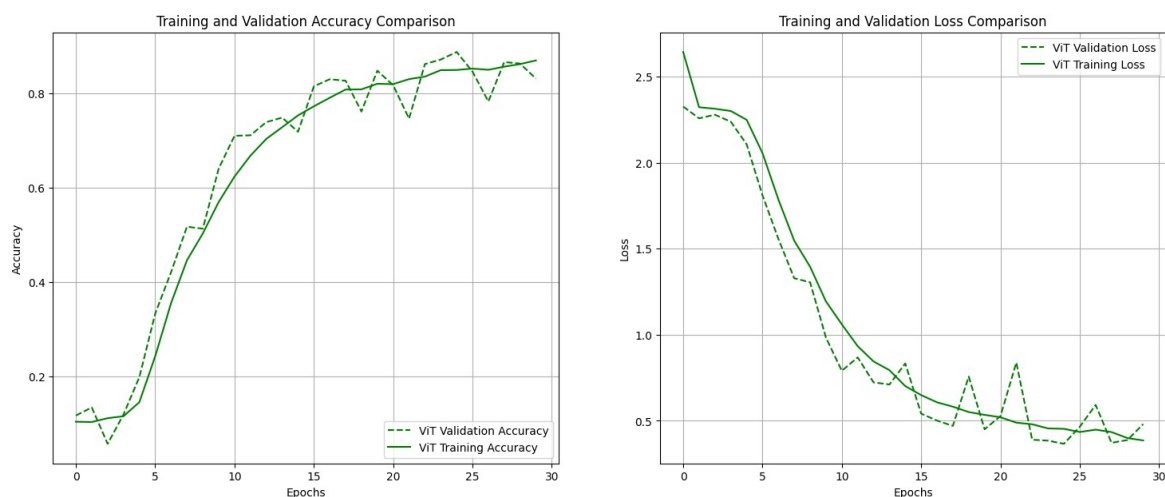
تغییر اندازه patch تأثیر قابل توجه و مستقیمی بر عملکرد مدل و هزینه محاسباتی دارد.

کاهش اندازه patch منجر به طولانی‌تر شدن دنباله ورودی برای ترنسفورمر می‌شود. ولی patch کوچک‌تر امکان مشاهده جزئیات ریزتر تصویر را فراهم می‌کنند که می‌تواند به دقت بالاتری بیانجامد. از طرف دیگر، دنباله طولانی‌تر مصرف حافظه و تعداد محاسبات را افزایش می‌دهد و آموزش مدل را بسیار کندتر می‌کند.

افزایش اندازه patch منجر به دنباله ورودی بسیار کوتاه‌تری می‌شود که جزئیات بیشتری را از دست می‌دهد. به همین جهت ویژگی‌های ریز ممکن است در یک تکه بزرگ گم شوند و دقت را کاهش دهند. اما دنباله کوتاه‌تر باعث می‌شود مدل سریع‌تر و با مصرف حافظه کمتر آموزش یابد.

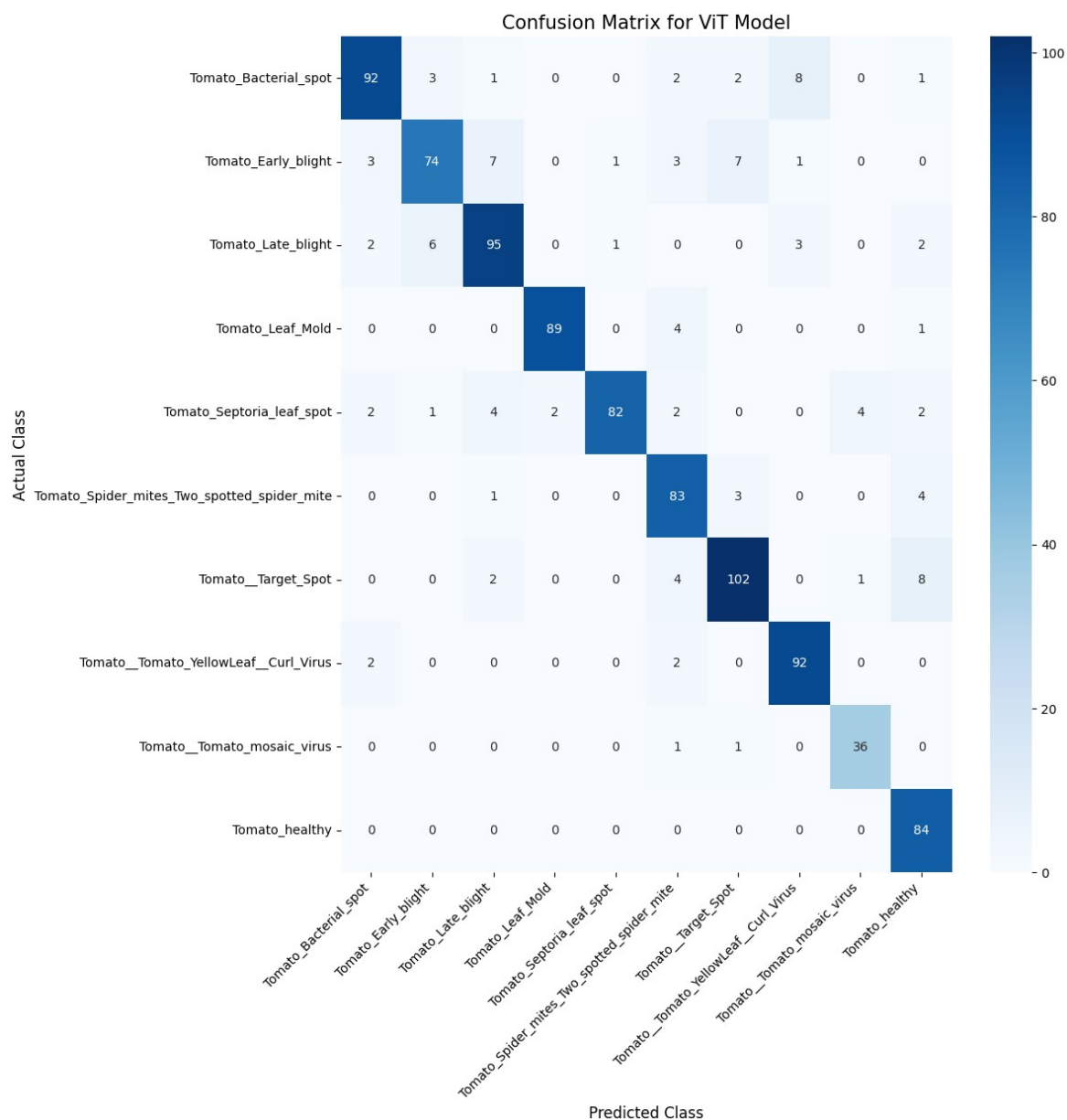
۱-۴-۳. آموزش مدل ViT

مدل را همانند مدل های CNN به مدت ۳۰ epoch آموزش دادیم.



شکل 11: نمودار دقت و تابع هزینه مدل ViT

می‌توان مشاهده کرد که مدل به درستی مجموعه داده را یاد گرفته و دچار بیش‌برازش نشده است و موفق شده دقت ۸۸٪ روی مجموعه اعتبارسنجی کسب کند.



شکل 12: ماتریس آشفتگی ViT

از ماتریس آشفتگی مشخص است که خطاهای طبقه‌بندی مدل به صورت نسبتاً مساوی توزیع شده‌اند، اما حتی مدل ViT نیز در طبقه‌بندی کلاس "Tomato_Early_blight" با مشکل مواجه شده است.

۵-۱. تحلیل و نتیجه گیری

بعد از بررسی نتایج مدل ها جدول های زیر را برای مقایسه مدل های تشکیل می دهیم:

Overall Model Performance Comparison					
	Model	Accuracy	Precision (Macro Avg)	Recall (Macro Avg)	F1-Score (Macro Avg)
0	1. CNN (Pre-trained)	0.883173	0.900804	0.887579	0.891404
1	2. CNN (From Scratch)	0.938907	0.949674	0.938612	0.942098
2	3. ViT (From Scratch)	0.888532	0.890462	0.895111	0.890284

شکل 13: مقایسه عملکرد کلی مدل ها

Detailed Report: 1. CNN (Pre-trained)					
		precision	recall	f1-score	support
	Tomato_Bacterial_spot	0.88	0.91	0.89	109
	Tomato_Early_blight	0.77	0.92	0.83	96
	Tomato_Late_blight	0.88	0.88	0.88	109
	Tomato_Leaf_Mold	1.00	0.80	0.89	94
	Tomato_Septoria_leaf_spot	0.86	0.80	0.83	99
	Tomato_Spider_mites_Two_spotted_spider_mite	0.84	0.93	0.89	91
	Tomato__Target_Spot	0.81	0.88	0.84	117
	Tomato__Tomato_YellowLeaf__Curl_Virus	1.00	0.90	0.95	96
	Tomato__Tomato_mosaic_virus	1.00	0.95	0.97	38
	Tomato_healthy	0.97	0.92	0.94	84
	accuracy			0.88	933
	macro avg	0.90	0.89	0.89	933
	weighted avg	0.89	0.88	0.88	933

شکل 14: جزئیات عملکرد مدل CNN از پیش آموزش دیده

Detailed Report: 2. CNN (From Scratch)				
	precision	recall	f1-score	support
Tomato_Bacterial_spot	0.94	0.96	0.95	109
Tomato_Early_blight	0.95	0.76	0.84	96
Tomato_Late_blight	0.85	0.95	0.90	109
Tomato_Leaf_Mold	0.98	0.96	0.97	94
Tomato_Septoria_leaf_spot	0.97	0.93	0.95	99
Tomato_Spider_mites_Two_spotted_spider_mite	0.99	0.91	0.95	91
Tomato__Target_Spot	0.86	0.98	0.92	117
Tomato__Tomato_YellowLeaf__Curl_Virus	0.99	0.98	0.98	96
Tomato__Tomato_mosaic_virus	1.00	0.95	0.97	38
Tomato_healthy	0.98	1.00	0.99	84
accuracy			0.94	933
macro avg	0.95	0.94	0.94	933
weighted avg	0.94	0.94	0.94	933

شکل 15: جزییات مدل CNN بدون آموزش قبلی

Detailed Report: 3. ViT (From Scratch)				
	precision	recall	f1-score	support
Tomato_Bacterial_spot	0.91	0.84	0.88	109
Tomato_Early_blight	0.88	0.77	0.82	96
Tomato_Late_blight	0.86	0.87	0.87	109
Tomato_Leaf_Mold	0.98	0.95	0.96	94
Tomato_Septoria_leaf_spot	0.98	0.83	0.90	99
Tomato_Spider_mites_Two_spotted_spider_mite	0.82	0.91	0.86	91
Tomato__Target_Spot	0.89	0.87	0.88	117
Tomato__Tomato_YellowLeaf__Curl_Virus	0.88	0.96	0.92	96
Tomato__Tomato_mosaic_virus	0.88	0.95	0.91	38
Tomato_healthy	0.82	1.00	0.90	84
accuracy			0.89	933
macro avg	0.89	0.90	0.89	933
weighted avg	0.89	0.89	0.89	933

شکل 16: جزییات عملکرد مدل ViT

از جدول‌های بالا می‌بینیم که مدل CNN که از ابتدا بدون وزن‌های آماده آموزش دید بهترین عملکرد را با دقت ۹۳٫۸٪ (و F1-Score برابر ۰٫۹۴) داشته است. مدل CNN از پیش‌آموزش‌دیده و مدل ViT عملکرد مشابهی با حدود ۹۰٪ دقت (و F1-Score برابر ۰٫۸۹) داشتند.

با توجه به اینکه مدل InceptionV3 دو برابر مدل ViT پارامتر دارد (۲۲ میلیون در مقابل ۱۱ میلیون)، برای کاربردهای سبک‌تر مانند اپلیکیشن موبایلی که در مقاله پیشنهاد شده، استفاده از مدل ViT منطقی‌تر است.

علاوه بر این استفاده از ViT و صرف کردن وقت بیشتر برای آموزش / Tune کردن آن به صرفه‌تر است چرا که ممکن است مدل ViT با آموزش بیشتر (اپوک‌های بیشتر) دقت بالاتری کسب کند؛ در حالی که این مسئله درباره مدل CNN صدق نمی‌کند، زیرا دقت اعتبارسنجی‌اش در انتهای آموزش کاهش یافت و نشانه‌های Overfit را نشان داد.

راه دیگر برای بهبود مدل‌ها و به‌ویژه ViT، استفاده از داده‌های بیشتر یا افزایش داده (data augmentation) شدیدتر است، چرا که برخلاف مدل‌های CNN، ترنسفورمرها درک ذاتی از اشکال مختلف ندارند و داده‌های بیشتر می‌تواند به یادگیری دقیق‌تر آن‌ها کمک کند.

پیش ۲ – Robust Zero-Shot Classification

۲-۰. مقدمه:

در این سؤال قصد داریم روش‌های مختلف حملات (adversarial attacks) را پیاده‌سازی کرده و آن‌ها را با یکدیگر مقایسه کنیم. برای این کار از کاری که در مقاله ارائه شده استفاده خواهیم کرد.

۲-۱. آشنایی با مدل CLIP، طبقه بندی تک ضرب و حملات خصمانه

۲-۱-۲. FGSM و PGD

قبل از توضیح FGSM و PGD، ابتدا توضیح مختصری درباره حملات خصمانه می‌دهیم:

حملات خصمانه (Adversarial attacks):

طبق توضیح مقاله، حمله خصمانه به تصویر اصلی یک آشفتگی بسیار کوچک و غیرقابل تشخیص اضافه می‌کند تا تصویری جدید بسازد که هدف آن بیشینه‌سازی خطا یا Loss مدل است.

Fast Gradient Sign Method (FGSM):

(براساس این توضیح در TensorFlow:

https://www.tensorflow.org/tutorials/generative/adversarial_fgsm)

FGSM یک روش سریع برای تولید مثال‌های خصمانه است. این روش با بهره‌گیری از نحوه رفتار تابع Loss مدل، با تنها یک تغییر در هر پیکسل تصویر ورودی، Loss مدل را بیشینه می‌کند. مراحل کار این روش به صورت زیر است:

1. محاسبه گرادیان: ابتدا گرادیان تابع Loss مدل را نسبت به هر پیکسل تصویر ورودی محاسبه می‌کند تا جهت افزایش بیشترین Loss مشخص شود.

2. به دست آوردن علامت: به جای استفاده مستقیم از مقدار گرادیان، فقط علامت هر عنصر گرادیان استفاده می‌شود؛ یعنی برای هر پیکسل مشخص می‌کند که مقدار آن باید افزایش (+1) یا کاهش (-1) یابد.

3. اعمال آشفته‌گی: بردار علامت در یک عدد اسکالر کوچک (اپسلون) ضرب می‌شود تا بزرگی کلی آشفته‌گی کنترل شود.
فرمول محاسبه FGSM:

$$adv_x = x + \epsilon * \text{sign}(\nabla_x J(\theta, x, y))$$

شکل 17: فرمول محاسبه FGSM

Projected Gradient Descent (PGD):

(براساس این توضیح در Medium:

<https://medium.com/@zachariaharungeorge/unveiling-the-power-of-projected-gradient-descent-in-adversarial-attacks-2f92509dde3c>)

PGD توسعه ایده اصلی حمله خصمانه است. به این صورت که به جای یک گام بزرگ، PGD با چندین گام کوچک‌تر عمل می‌کند تا شانس یافتن یک آشفته‌گی موفق را افزایش دهد. مراحل کار:

1. مقداردهی اولیه: با یک گام کوچک و تصادفی از تصویر اصلی دور می‌شود، اما همچنان در محدوده مجاز باقی می‌ماند.

2. حلقه: به تعداد از پیش تعیین شده ای، گرادیان Loss نسبت به پیکسل‌های تصویر فعلی محاسبه می‌شود و سپس یک گام کوچک به سمت علامت گرادیان برداشته می‌شود. اندازه این گام با پارامتر α مشخص می‌شود (مثلاً $\alpha = 1/255$ در مقاله).

3. پروجکشن: پس از هر گام، اگر تصویر جدید از محدوده ϵ پیرامون تصویر اصلی خارج شده باشد، با یک عملیات Clipping آشفته‌گی به گونه‌ای محدود می‌شود که مقدار آن از ϵ تجاوز نکند.

از آنجا که PGD یک فرایند تکراری (iterative) است و منطقه پیرامون تصویر را با دقت بیشتری جستجو می‌کند، حمله‌ای قوی‌تر از FGSM محسوب می‌شود و یکی از معیارهای استاندارد برای ارزیابی مقاومت مدل در برابر حملات خصمانه است.

۲-۱-۲. CLIP

۲-۱-۲-۱. معماری CLIP

ایده اصلی CLIP (Contrastive Language-Image Pre-training) یادگیری یک فضای Multi modal است که در آن تصاویر و متن‌های مرتبط از نظر معنایی در مجاورت یکدیگر قرار می‌گیرند. برای دستیابی به این هدف، مدل از دو بخش اصلی تشکیل شده است: یک encoder تصویر و یک encoder متن.

encoder تصویر:

این بخش تصویر ورودی را دریافت کرده و یک بردار ویژگی خروجی می‌دهد. در مقاله معرفی‌کننده CLIP، دو معماری مختلف برای این encoder در نظر گرفته شدند:

- نسخه تغییر یافته ResNet-50

- شامل بهبودهای ResNet-D

- استفاده از antialiased rect-2 blur pooling

- جایگزینی لایه global average pooling با مکانیسم attention pooling

- معماری Vision Transformer (ViT)

- پردازش تصویر به صورت پچ‌های مسطح شده.

encoder متن:

این بخش متن ورودی را دریافت کرده و بردار ویژگی متناظر را تولید می‌کند. معماری مورد استفاده یک ترنسفورمر ۶۳ میلیون پارامتری با ۱۲ لایه و ۸ سر attention است.

هر دو encoder به طور مشترک و از ابتدا آموزش داده می‌شوند تا خروجی‌هایشان را در یک فضای مشترک قرار دهند. خلاصه‌ای از معماری مدل در شکل زیر نشان داده شده است.

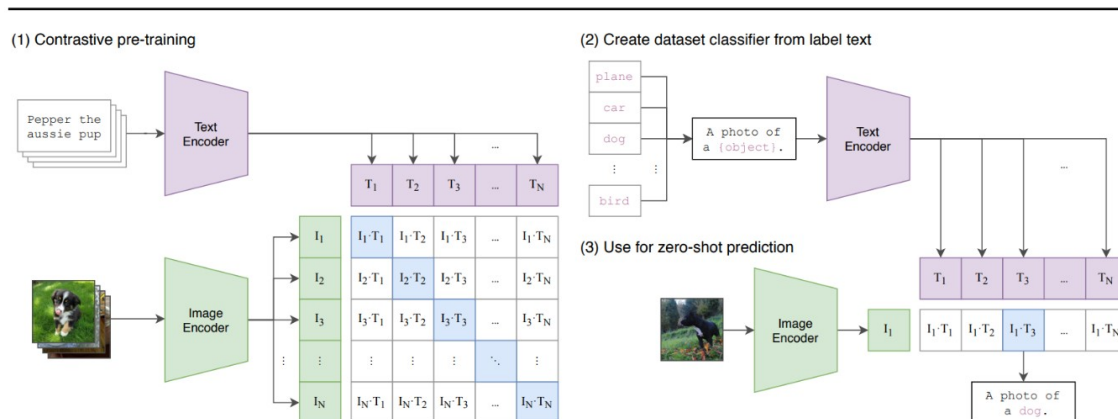


Figure 1. Summary of our approach. While standard image models jointly train an image feature extractor and a linear classifier to predict some label, CLIP jointly trains an image encoder and a text encoder to predict the correct pairings of a batch of (image, text) training examples. At test time the learned text encoder synthesizes a zero-shot linear classifier by embedding the names or descriptions of the target dataset's classes.

شکل 18: خلاصه معماری CLIP

۲-۱-۲-۲. آموزش Contrastive

به جای استفاده از روش های سنتی مانند پیش بینی دقیق کپشن متن برای یک تصویر، CLIP از آموزش Contrastive استفاده می کند. هدف تولید محتوا نیست، بلکه تطبیق تصاویر و متن های موجود درون یک دسته بزرگ است.

فرایند شامل مراحل زیر است:

• ایجاد یک Batch

• encode همه تصاویر و متن ها

• محاسبه شباهت: با استفاده از Cosine Similarity بین هر تصویر و هر متن encode شده شباهت را محاسبه می کند و یک ماتریس N در N از مقادیر شباهت ها می سازد

• Contrastive Goal: در قطر این ماتریس، مقادیر شباهت برای جفت های درست (تصویر، متن) قرار دارد و سایر درایه ها مربوط به جفت های نادرست هستند. هدف آموزش مدل این است که شباهت N جفت درست را بیشینه و شباهت بقیه جفت ها را کمینه کند. به طور خلاصه، مدل می آموزد که تصویر را با کپشن درستش منطبق کند.

۲-۱-۲-۳. تابع خطا

مدل از یک تابع خطای متقارن مبتنی بر کراس-انترپولی (Symmetric Cross-Entropy Loss) استفاده می کند که در قطعه کد زیر مربوط به مقاله معرفی CLIP آمده است.

```

# image_encoder - ResNet or Vision Transformer
# text_encoder - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l] - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T) #[n, d_t]

# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss = (loss_i + loss_t)/2

```

Figure 3. Numpy-like pseudocode for the core of an implementation of CLIP.

شکل 19: شبه کد پیاده سازی *loss* برای CLIP

«مقارن» بودن این تابع به این معنی است که خطا از دو دیدگاه مختلف محاسبه و سپس میانگین گرفته می‌شود:

- خطای تصویر به متن ($loss_i$): برای هر تصویر، مدل باید در میان متن‌های موجود، متن درست را تشخیص دهد. این کار به صورت مسأله طبقه‌بندی با استفاده از کراس-انتروپی عادی انجام می‌شود.

- خطای متن به تصویر ($loss_t$): برای هر متن، مدل باید در میان تصاویر موجود، تصویر درست را تشخیص دهد. این هم با کراس-انتروپی محاسبه می‌شود.

خطای نهایی دسته برابر است با میانگین این دو خطا:

$$\text{loss} = (\text{loss}_i + \text{loss}_t) / 2$$

۲-۱-۳. طبقه بندی تک ضرب در مقابل طبقه بندی معمولی

تفاوت اصلی در نحوه آموزش مدل‌ها و قابلیت‌های پیش‌بینی آن‌هاست.

طبقه بندی معمولی (نظارت‌شده):

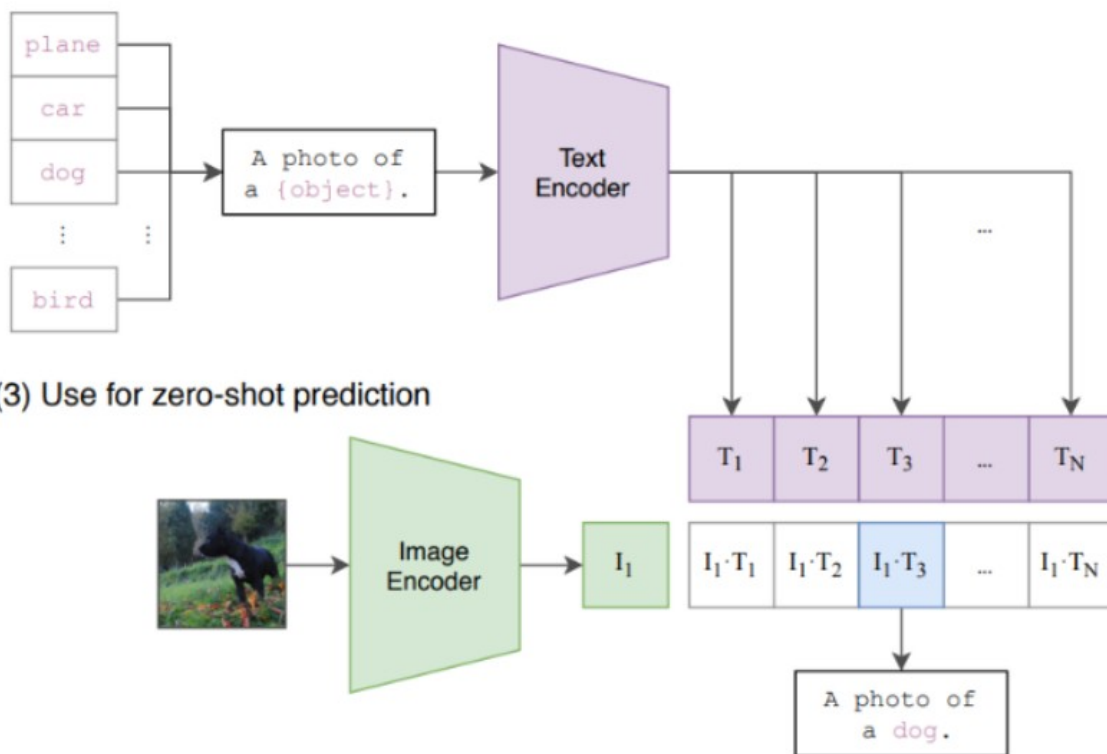
مدل روی مجموعه‌ای ثابت از برچسب‌ها آموزش داده می‌شود. برای شناسایی یک دسته جدید باید مدل را مجدداً آموزش داد یا فاین‌تیون کرد و داده‌های برچسب‌خورده جدید فراهم کرد. خروجی معمولاً یک لایه softmax است که از میان کلاس‌های دیده‌شده بهترین گزینه را انتخاب می‌کند.

طبقه بندی تک ضرب

مدل به جای استفاده از برچسب‌های ثابت، رابطه‌ای بین ویژگی‌های تصویری و توصیف‌های زبانی یاد می‌گیرد. این روش این امکان را می‌دهد که بدون آموزش مجدد روی دسته‌های جدید، تصاویر را به کلاس‌های نادیده‌شده نسبت دهد. در واقع یک طبقه‌بند جدید از زبان طبیعی ساخته می‌شود.

در CLIP این کار به صورت زیر انجام می‌شود:

(2) Create dataset classifier from label text



شکل 20: طبقه بندی تک ضرب در CLIP

(2) ساخت طبقه‌بند از متن

• ابتدا نام تمام کلاس‌های هدف (مثلاً «هواپیما»، «ماشین»، «سگ») مشخص می‌شود.

• این نام‌ها در قالب‌هایی مانند «عکسی از یک {کلاس}.» قرار می‌گیرند.

• هر قالب به encoder متن CLIP فرستاده می‌شود تا بردار ویژگی متن (text embedding) تولید شود.

• این بردارها به عنوان وزن‌های یک طبقه‌بند خطی تک ضرب در نظر گرفته می‌شوند.

(3) پیش‌بینی تک ضرب

• تصویر ورودی به encoder تصویر CLIP داده می‌شود تا بردار ویژگی تصویر (image embedding) تولید شود.

• شباهت کسینوسی بین بردار تصویر و همه بردارهای متن محاسبه می‌شود.

• برچسب متن با بالاترین شباهت کسینوسی، خروجی نهایی مدل است.

بدین ترتیب CLIP بدون نیاز به آموزش مجدد می‌تواند روی داده‌های دلخواه طبقه‌بندی انجام دهد.

۲-۱-۴. انواع حملات خصمانه

White-Box Attacks

در این حالت، مهاجم به‌طور کامل به اطلاعات داخلی مدل دسترسی دارد. این اطلاعات شامل کد، معماری مدل، پارامترها و تابع خطای مورد استفاده هنگام آموزش است. به‌دلیل این دسترسی کامل، مهاجم می‌تواند از روش‌های مبتنی بر گرادیان مانند FGSM و PGD استفاده کند تا دقیقاً اثربخش‌ترین اغتشاش را برای فریب مدل محاسبه و اعمال کند.

Black-Box Attacks

در این حالت، مهاجم هیچ دسترسی‌ای به ساختار داخلی مدل ندارد و مدل را به‌صورت «جعبه سیاه» در نظر می‌گیرد. مهاجم فقط می‌تواند ورودی‌ای به مدل بدهد و خروجی آن (برچسب پیش‌بینی‌شده و confidence score) را مشاهده کند. از آنجا که مهاجم نمی‌تواند گرادیان‌های مدل را مستقیم محاسبه کند، باید از روش‌های دیگری مانند حملات مبتنی بر کوئری (با ارسال تعداد زیادی درخواست به مدل برای استنباط decision boundary) یا حملات انتقالی (ترنسفر) استفاده کند.

جدول ۲: مقایسه روش‌های White-Box و Black-Box

Aspect	White-Box	Black-Box
اطلاعات	دسترسی کامل به معماری، پارامترها و گرادیان‌ها	صرفاً دسترسی به ورودی و خروجی
اثربخشی	بسیار مؤثر، نمایانگر بدترین حالت برای آزمایش مقاومت مدل	کمتر مؤثر، نرخ موفقیت بستگی زیادی به روش حمله دارد
روش	عمدتاً از روش‌های مبتنی بر گرادیان مانند FGSM و PGD استفاده می‌کند	متکی بر روش‌های مبتنی بر کوئری یا انتقال برای دور زدن نیاز به گرادیان

۲-۱-۵. حملات انتقالی

مطابق این پست در [Medium](#)، گرچه حملات White-Box علیه یک مدل خاص قدرتمندتر هستند، اما کاربرد آن‌ها در دنیای واقعی محدود است. در مقابل، حملات انتقالی تهدید جدی‌تر و عملی‌تری محسوب می‌شوند، به دلایل زیر:

۱. سیستم‌های واقعی جعبه سیاه هستند

در بیشتر موارد، مدل‌های مورد استفاده در شرکت‌ها محرمانه‌اند و مهاجمان به معماری، پارامترها یا گرادین‌های آن‌ها دسترسی ندارند. بنابراین حمله White-Box در عمل امکان‌پذیر نیست. حملات انتقالی راهی برای نفوذ به این سیستم‌ها فراهم می‌کنند.

۲. عدم نیاز به جزئیات مدل

در یک حمله انتقالی، مهاجم می‌تواند ابتدا مدل خود را (که برای او White-Box است) آموزش دهد و سپس با استفاده از روش‌های White-Box مانند PGD یک مثال خصمانه تولید کند که مدل را فریب می‌دهد. همان مثال خصمانه سپس می‌تواند برای حمله به مدل هدف جعبه سیاه استفاده شود.

۳. آسیب‌پذیری‌های عمومی

دلیل اصلی کارآمدی حملات انتقالی این است که مثال‌های خصمانه معمولاً بر ویژگی‌ها یا ضعف‌های بنیادین یادگیری شبکه‌های عصبی تکیه دارند و نه وزن‌های خاص یک مدل. مدل‌های مختلف آموزش‌دیده برای وظایف مشابه اغلب نمایش‌های ویژگی مشابهی یاد می‌گیرند و در نتیجه آسیب‌پذیری‌های مشابهی دارند. با توجه به مقاله تحقیقات نشان داده‌است که ویژگی‌های حملات خصمانه از مدلی به مدل دیگر و حتی از یک وظیفه به وظیفه‌ای دیگر قابل انتقال هستند.

۲-۱-۶. LoRA

LoRA که مخفف Low-Rank Adaptation است، یک روش بهینه برای fine-tune مدل‌های زبانی بزرگ بر پایه این فرض است که تغییرات وزن‌های مدل در طول fine-tune کردن دارای low intrinsic rank است و می‌توان آن‌ها را با تعداد پارامترهای بسیار کمتری نمایش داد. به جای fine tune تمام پارامترهای مدل از ابتدا، LoRA مراحل زیر را انجام می‌دهد:

۱. منجمد (freeze) کردن وزن‌های از پیش‌آموزش‌دیده

بیشتر وزن‌های اصلی مدل ثابت نگه داشته می‌شوند و تغییر نمی‌کنند.

۲. اضافه کردن ماتریس‌هایی با رتبه کم

برای هر ماتریس وزن W_0 در مدل، LoRA دو ماتریس با رتبه کوچک‌تر و قابل‌آموزش به نام‌های A و B اضافه می‌کند.

۳. یادگیری ماتریس‌های اضافه‌شده

تنها ماتریس‌های A و B آموزش می‌یابند. حاصل ضرب BA برابر تغییر ΔW روی وزن اصلی است و خروجی نهایی برابر $W_0x + BAx$ خواهد بود.

۴. ادغام برای استفاده در مرحله استنتاج (inference)

پس از آموزش، ماتریس BA را با W_0 جمع می‌کنند ($W = W_0 + BA$) تا وزن‌های نهایی به دست آیند. در نتیجه در زمان استنتاج تعداد پارامترها و معماری مدل تغییر نمی‌کند.

خلاصه این فرآیند در شکل زیر قابل مشاهده است:

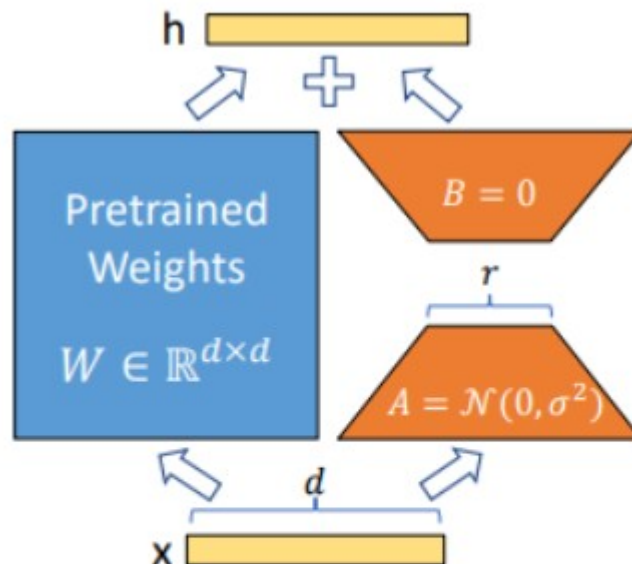


Figure 1: Our reparametrization. We only train A and B .

شکل 21: خلاصه عملکرد LoRA

سه دلیل اصلی برای استفاده از LoRA

۱. بهره‌وری بالا در استفاده از پارامترها

LoRA تعداد پارامترهای قابل آموزش را تا $10,000$ برابر کاهش می‌دهد و حجم چک‌پوینت را از صدها گیگابایت به چند مگابایت می‌رساند.

۲. عدم افزایش تأخیر (delay) در استنتاج

چون ماتریس‌های BA پس از آموزش با وزن‌های اصلی ادغام می‌شوند، مدل هیچ پارامتر یا لایه اضافی ندارد و پاسخ‌ها با همان سرعت مدل قبلی تولید می‌شوند.

۳. آموزش سریع‌تر و مقرون‌به‌صرفه‌تر

در مقایسه با fine tuning کامل، LoRA در GPT-3 175B مصرف حافظه را سه برابر کاهش داده و تا ۲۵٪ سرعت آموزش را افزایش می‌دهد، چرا که گرادینان برای اکثر پارامترها محاسبه نمی‌شود.

۲-۱-۷. دو مقاله که CLIP را گسترش می‌دهند

۲-۱-۷-۱. مقاله اول: Post-pre-training for Modality Alignment in Vision-Language Foundation Models

این مقاله روش CLIP-Refine را معرفی می‌کند که پس از پیش‌آموزش (Pre-training) اولیه و قبل از هرگونه فاین‌تیون، برای بهبود عملکرد مدل‌های CLIP موجود طراحی شده است. هدف اصلی رفع اختلاف مدالیت (Modality gap) یعنی اختلاف در چگونگی خوشه‌بندی ویژگی‌های تصویر و متن در فضای Embedding است. در این روش دو تکنیک جدید به کار رفته است:

- Random Feature Alignment (RaFA) برای هم‌تراز کردن تصادفی توزیع ویژگی‌های تصویر و متن
- Hybrid Contrastive-Distillation (HyCD) که با ترکیب برچسب‌های واقعی و دانش مدل CLIP اصلی، اطلاعات جدید یاد می‌گیرد بدون آنکه آنچه قبلاً آموخته شده را فراموش کند.

این رویکرد نیازمند حداقل آموزش است و تنها یک اپوک روی مجموعه داده‌های کوچک کفایت می‌کند. تابع خطای CLIP-Refine به‌طور قابل‌توجهی عملکرد تک ضرب CLIP را در وظایف مختلف افزایش می‌دهد. با کاهش شکاف مدالیت و یکنواخت‌سازی ویژگی‌ها، مدل توانایی بهتری در تعمیم به داده‌های نادیده پیدا می‌کند. در آزمایش‌ها نشان داده شده که CLIP-Refine در ۱۲ مجموعه داده دسته‌بندی، میانگین دقت را از ۵۲.۷۴٪ به ۵۴.۶۹٪ افزایش می‌دهد.

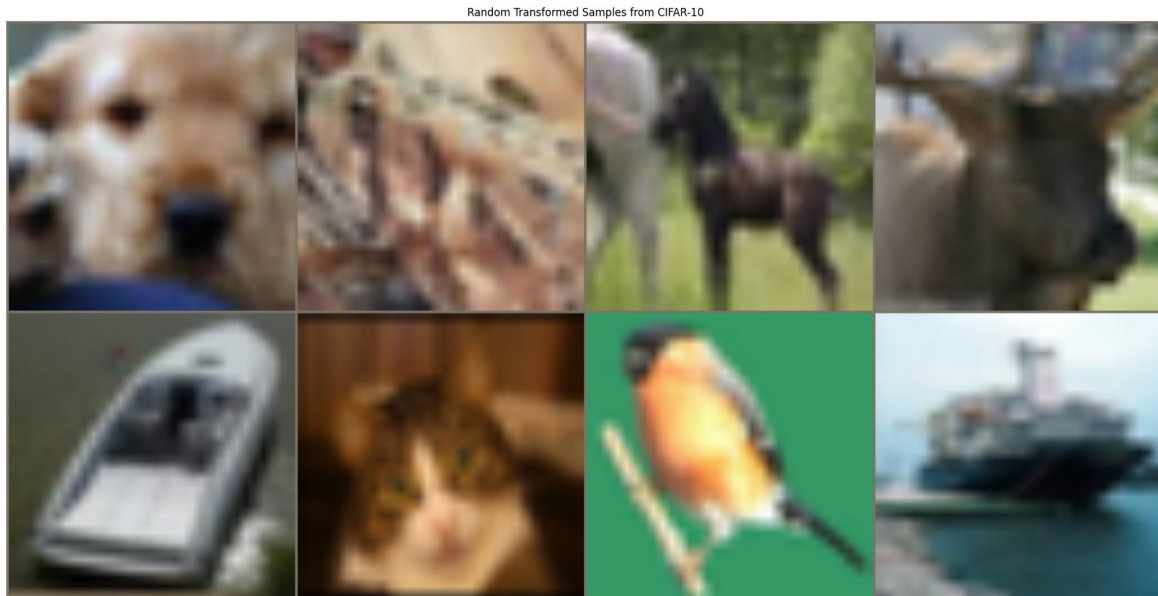
این مقاله روش FairerCLIP را معرفی می‌کند که هدفش کاهش تعصبات در پیش‌بینی‌های تک ضرب مدل‌های CLIP است. این تعصبات می‌توانند از وابستگی‌های درون‌زا (Intrinsic dependencies) (مثل همبستگی جنسیت با ویژگی‌های چهره) یا همبستگی‌های تصادفی (مثل پس‌زمینه تصویر) ناشی شوند. روش FairerCLIP برای حذف تعصب از فضای هیلبرت (Reproducing Kernel Hilbert Space (RKHS) استفاده می‌کند. این رویکرد امکان تعریف یک تابع هدف صریح برای کاهش وابستگی آماری به ویژگی‌هایی مانند نژاد یا جنسیت را فراهم می‌کند در حالی که اطلاعات مربوط به ویژگی هدف را حفظ کرده و هم‌ترازی تصویر و متن را نیز نگه می‌دارد. مزیت کلیدی این روش انعطاف‌پذیری آن است؛ می‌توان آن را با یا بدون برچسب‌های واقعی آموزش داد و به دلیل استفاده از راه‌حل‌های closed-form، بسیار کارآمد است.

روش FairerCLIP به‌طور مؤثری تعصب را کاهش می‌دهد در حالی که دقت دسته‌بندی بدون شات CLIP را حفظ یا حتی افزایش می‌دهد. با منصفانه‌تر کردن پیش‌بینی‌ها و کاهش اختلاف عملکرد میان گروه‌های مختلف، قابلیت اطمینان و مقاومت مدل در سناریوهای تک ضرب دشوار افزایش می‌یابد. برای مثال، در مجموعه داده‌هایی با وابستگی‌های درون‌زا، FairerCLIP میزان EOD را تا نزدیک صفر کاهش داد در حالی که دقت دسته‌بندی پایین نیامد. همچنین در مجموعه داده‌هایی با همبستگی‌های تصادفی، بدترین دقت گروه را بهبود داد و شکاف عملکرد بین گروه‌های مختلف را کاهش داد.

۲-۲. پیاده‌سازی و مقایسه روش‌های آموزش خصمانه

۲-۲-۱. آماده‌سازی داده‌ها

در این بخش مجموعه داده CIFAR-۱۰ را با استفاده از torchvision دانلود می‌کنیم، آن را به مجموعه‌های آموزش، اعتبارسنجی و آزمون تقسیم می‌کنیم، برای استفاده در CLIP تبدیل می‌نماییم و یک دسته نمونه تصادفی از داده‌ها را نمایش می‌دهیم.



شکل 22: نمونه تصاویر CIFAR-10
چون تصاویر CIFAR-10 ابعاد 32×32 دارند به همین جهت وقتی در ابعاد بزرگتر نشان داده می‌شوند به نظر تار می‌آیند!

۲-۲-۲. آماده‌سازی مدل

در این بخش مدل CLIP پیش‌آموزش‌دیده را از کتابخانه Hugging Face Transformers بارگیری می‌کنیم، مدل ResNet-۲۰ پیش‌آموزش‌دیده برای CIFAR-۱۰ را از PyTorch Hub دانلود می‌نماییم، هر دو مدل را در حالت ارزیابی (eval) قرار می‌دهیم، قالب‌های متنی توصیفی بر اساس نام کلاس‌های CIFAR-۱۰ ایجاد می‌کنیم و از CLIP برای پردازش این قالب‌ها و تولید بردارهای ویژگی متنی لازم برای دسته‌بندی بدون شات استفاده می‌نماییم.

۲-۲-۳. آزمایش CLIP، حمله PGD و بصری‌سازی

در این بخش ابتدا CLIP را روی داده‌های پاک ارزیابی می‌کنیم، سپس با پارامترهای مشخص شده حمله PGD را اجرا کرده و دوباره CLIP را روی داده‌های خصمانه ارزیابی می‌کنیم و تعدادی از نمونه‌های حمله را نمایش می‌دهیم.

2-2-3-2. Evaluating CLIP

```
[12] 1 clean_accuracy = evaluate_clip(clip_model, test_loader, text_features)
      2 print()
      3 print(f"CLIP's Zero-Shot Accuracy on clean CIFAR-10 images: {clean_accuracy:.2f}%")
```

Evaluating: 100% | 313/313 [00:28<00:00, 10.87it/s]
CLIP's Zero-Shot Accuracy on clean CIFAR-10 images: 87.83%

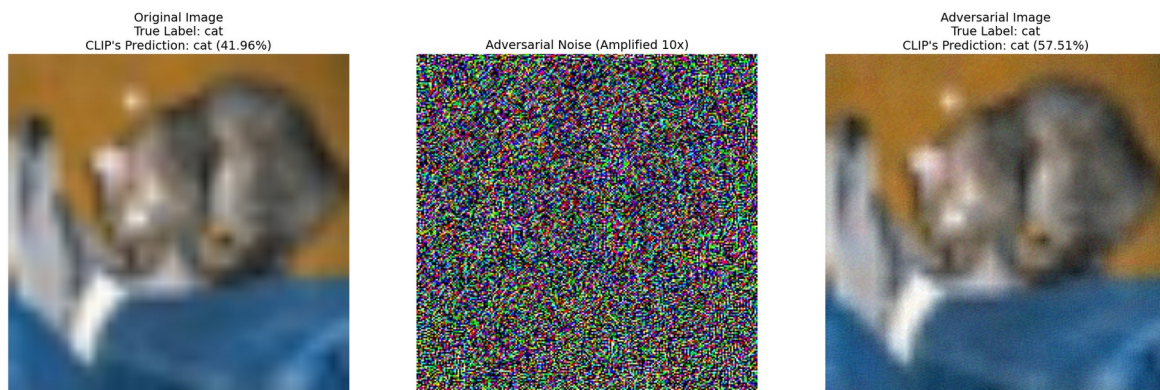
شکل 23: دقت CLIP روی داده های تمیز ۸۷.۸۳٪

2-2-3-3. PGD Attack

```
1 # !pip install torchattacks
2 import torchattacks
3
4 pgd_attack = torchattacks.PGD(target_model, eps=8/255, alpha=2/255, steps=7)
5
6 pgd_attack.set_normalization_used(mean=[0.4914, 0.4822, 0.4465], std=[0.2023, 0.1994, 0.2010])
7
8 adversarial_accuracy = evaluate_clip(clip_model, test_loader, text_features, attack_model=pgd_attack)
9 print()
10 print(f"CLIP's Zero-Shot Accuracy on adversarial images: {adversarial_accuracy:.2f}%")
```

Evaluating: 100% | 313/313 [06:04<00:00, 1.17s/it]
CLIP's Zero-Shot Accuracy on adversarial images: 84.56%

شکل 24: دقت CLIP روی داده های خصمانه ۸۴.۵۶٪



شکل 25: نمونه داده اصلی، داده خصمانه و نویز اضافه شده (برای نمایش بهتر نویز شدت آن ۱۰ برابر شده است)

LoRA .۲-۲-۴

در این بخش یک LoraConfig تعریف می‌کنیم تا لایه‌های بینایی (Vision) مدل CLIP را تطبیق دهیم، مدل CLIP را با پیکربندی LoRA wrap می‌کنیم تا یک مدل جدید ایجاد شود، یک epoch مدل را آموزش می‌دهیم که در این آموزش برای هر دسته نمونه‌های خصمانه را تولید کرده و تنها وزن‌های LoRA را به‌روز می‌کنیم و در نهایت مدل جدید را دوباره روی داده‌های پاک و خصمانه ارزیابی می‌کنیم تا تغییر در مقاومت آن را بسنجیم.

```
Initial Clean Accuracy: 87.83%
Initial Adversarial Accuracy: 84.56%

After LoRA Adversarial Training:
Clean Accuracy: 94.95%
Adversarial Accuracy: 96.82%
```

شکل 26: دقت بعد از Fine-tuning با LoRA

همان‌طور که دیده می‌شود دقت مدل پس از فاین‌تیون به‌طور قابل‌توجهی بهبود یافته است. پس از این آموزش، مدل حتی روی حملات خصمانه بهتر از تصاویر عادی عمل می‌کند!

البته باید توجه داشت که روش ما کمی با آنچه در مقاله آمده متفاوت است. در مقاله آموزش خصمانه روی مجموعه داده‌ای انجام شده که CLIP روی آن از پیش آموزش دیده (ImageNet اگر اشتباه نکنم) و سپس دفاع در برابر حملات خصمانه روی سایر dataset ها و به صورت تک ضرب آزمایش شده است. اما به دلیل محدودیت‌های سخت‌افزاری و زمانی، ما مدل را روی همان مجموعه داده‌ای که تست می‌کردیم فاین‌تیون کردیم، بنابراین دیگر چندان تک ضرب به حساب نمی‌آید! (با این حال حتی روی این مجموعه داده نسبتاً کوچک‌تر هم فرایند بسیار زمان‌بر بود!)

۲-۲-۵. TeCoA

در این بخش پیاده‌سازی TeCoA (Text-guided Contrastive Adversarial Training) را مطابق توضیحات ارائه شده در بخش‌های قبلی انجام می‌دهیم. پس از آموزش مجدد (فاین‌تیون) مدل با این تابع خطا، عملکرد آن را مجدداً مشابه آنچه در بخش قبل انجام دادیم ارزیابی خواهیم کرد.

```
➡ Evaluating: 100%|██████████| 313/313 [00:32<00:00, 9.60it/s]
TeCoA-tuned CLIP's accuracy on clean images: 95.70%
Evaluating: 100%|██████████| 313/313 [06:09<00:00, 1.18s/it]
TeCoA-tuned CLIP's accuracy on adversarial images: 97.22%
```

شکل 27: دقت بعد از Fine-tuning با TeCoA

پس از فاین تیون مدل با TeCoA، دقت روی تصاویر عادی و متخاصم نسبت به LoRA بهبود یافت.

۲-۲-۶. مقایسه و ارزیابی

```
➡ Summary of All Models
Initial Model Performance:
  Clean Accuracy: 87.83%
  Adversarial Accuracy: 84.56%
After Standard Adversarial Training (LoRA + CrossEntropy):
  Clean Accuracy: 94.95%
  Adversarial Accuracy: 96.82%
After TeCoA Training (LoRA + Contrastive Loss):
  Clean Accuracy: 95.70%
  Adversarial Accuracy: 97.22%
```

شکل 28: مقایسه عملکرد هر دو روش

با نگاهی به دقت مدل پس از هر کدام از فاین تیون ها، می توان دید استفاده از TeCoA بهترین بهبود را داشته اس : دقت روی داده های پاک به میزان ۸٪ (از ۸۷٫۸۳٪ به ۹۵٫۷۰٪) و دقت روی داده های خصمانه به میزان ۱۳٪ (از ۸۴٫۵۶٪ به ۹۷٫۲۲٪) افزایش داشته است.

با این حال باید توجه داشت طبق مقاله، مزیت واقعی TeCoA در دفاع در برابر حملات خصمانه تک ضرب ظاهر می شود. یعنی وقتی مدل روی یک مجموعه داده آموزش دیده و روی مجموعه داده ای متفاوت (به صورت تک ضرب) مورد حمله قرار می گیرد، چیزی که متأسفانه به دلیل محدودیت های زمانی و سخت افزاری نتوانستیم آن را آزمایش کنیم!

به طور کلی ولی با توجه به نتایج مقاله (و نتایج بدست آمده در این بخش) استفاده از روش TeCoA برای LoRA و نزدیک‌تر کردن فضای کلمات به فضای تصاویر عمل‌کرد بهتری هم در نمونه‌های عادی و هم در نمونه‌های خصمانه دارد.